AIA7: AI Agents and Autonomous Systems

Module Introduction and Theoretical Foundations

AI Agents represent the next paradigm beyond static LLMs: **autonomous systems** that perceive, plan, act, reflect, and adapt in dynamic environments. This module covers agent architectures, planning algorithms, tool integration, and memory systems—essential for building production agents like AutoGPT, BabyAGI, and enterprise automation.

Theoretical Foundations:

1. **LLM-as-Agent**: Reasoning → Action loop
2. **POMDP Formulation**: Partially observable environments
3. **Monte Carlo Tree Search**: LLM-powered planning
4. **Multi-Agent Systems**: Cooperation/competition dynamics

Agent Capability Spectrum:

text

Reactive (ReAct) → Planning (ToT) → Hierarchical → Multi-Agent

(Figure 1: Agent performance vs environment complexity)

Section 1: Agent Frameworks and Core Loop

ReAct (Yao 2022) - Reasoning + Acting:

text

Thought → Action → Observation → repeat

Core Agent Loop:

python

```python
class BaseAgent:
    def __init__(self, llm, tools, memory):
        self.llm = llm
        self.tools = {tool.name: tool for tool in tools}
        self.memory = memory

    def run(self, task: str, max_steps: int = 20) -> str:
        self.memory.add("task", task)

        for step in range(max_steps):
```

```
        # Observe state
        context = self.memory.retrieve_recent()

        # Reason + plan
        prompt = self.build_react_prompt(context, task)
        response = self.llm.generate(prompt)

        # Parse action
        if self.parse_action(response) == "finish":
            return self.memory.get_final_answer()

        # Execute
        observation = self.execute_action(response)
        self.memory.add("observation", observation)

    return "Max steps exceeded"

  def build_react_prompt(self, context, task):
    return f"""Task: {task}

Recent interactions:
{context}

Respond in format:
Thought: [your reasoning]
Action: [tool_name] {{"arg1": "value"}}
Observation: [result]

Or finish:
Final Answer: [solution]"""
```

Production Frameworks Comparison:

text

LangChain: Batteries included, complex

LlamaIndex: RAG-focused

CrewAI: Multi-agent orchestration

AutoGen: Microsoft research framework

Section 2: Planning Algorithms for Agents

Monte Carlo Tree Search (MCTS) + LLM:

text

1. Selection: UCT policy (exploration/exploitation)

2. Expansion: LLM generates actions

3. Rollout: Simulate trajectories

4. Backpropagate: Value estimates

Tree-of-Thoughts (ToT) Planning:

python

```python
class ToTPlanner:
    def __init__(self, llm, max_depth=4, branches=3):
        self.llm = llm
        self.max_depth = max_depth
        self.branches = branches

    def plan(self, problem):
        root = Node(problem)
        frontier = [root]

        while frontier:
            node = self.select_best(frontier)
            if self.is_solution(node):
                return node.value

            if node.depth < self.max_depth:
```

```
            children = self.llm_generate_actions(node.state, self.branches)
            for child in children:
                frontier.append(Node(child, parent=node))
            frontier.remove(node)  # BFS

        return self.best_solution(root)

    def llm_generate_actions(self, state, n):
        prompt = f"Given: {state}\nGenerate {n} concrete next steps:"
        return self.llm.generate(prompt)
```

Hierarchical Planning:

text

```
High-level: "Book flight + hotel"
Low-level: API calls, form filling
```

Section 3: Tool Usage and Function Calling

Tool Definition Schema:

python

```
tools = [
    {
        "name": "web_search",
        "description": "Search internet for current information",
        "parameters": {
            "type": "object",
            "properties": {"query": {"type": "string"}},
            "required": ["query"]
        }
    },
    {
        "name": "calculator",
        "description": "Solve math expressions",
```

```
        "parameters": {
            "type": "object",
            "properties": {"expression": {"type": "string"}},
            "required": ["expression"]
        }
    }
]
```

Function Calling Implementation:

python

```python
class ToolAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

    def execute(self, query):
        messages = [{"role": "user", "content": query}]

        while True:
            response = self.llm.chat(
                messages,
                tools=self.tools,
                tool_choice="auto"
            )

            if response.tool_calls:
                for tool_call in response.tool_calls:
                    tool_name = tool_call.function.name
                    args = json.loads(tool_call.function.arguments)

                    # Execute tool
                    result = self.tools[tool_name](**args)
```

```
            # Add observation
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": json.dumps(result)
            })
        else:
            return response.content
```

Tool Use Patterns:

text

1. Zero-shot function calling (OpenAI)
2. ReAct-style natural language tools
3. Multi-tool chaining

Section 4: Memory Systems for Long-Term Reasoning

Memory Hierarchy:

text

Short-term: KV cache (context window)

Long-term: Vector store (RAG)

Episodic: Past trajectories

Working: Current reasoning trace

Memory-Augmented Agent:

python

```
class MemoryAgent(BaseAgent):
    def __init__(self, llm, tools, vector_store):
        super().__init__(llm, tools, MemoryManager(vector_store))

class MemoryManager:
    def __init__(self, vector_store):
        self.store = vector_store
```

```
        self.episodic = []
        self.summary = ""

    def add(self, event_type, content):
        self.episodic.append({"type": event_type, "content": content})

        # Embed and store
        embedding = embedder.encode(content)
        self.store.add([embedding], metadata={"type": event_type})

        # Maintain summary
        if len(self.episodic) % 10 == 0:
            self.summary = self.summarize_recent()

    def retrieve_relevant(self, query, k=5):
        q_emb = embedder.encode(query)
        scores, indices = self.store.search(q_emb, k)
        return [self.store.metadata[i] for i in indices[0]]
```

Reflection and Self-Critique:

text

```
Critic Agent: "Rate this plan 1-10, suggest improvements"
Verifier: "Does this satisfy constraints?"
```

Section 5: Multi-Agent Systems and Orchestration

Agent Society Patterns:

text

```
1. Debate: Multiple perspectives → synthesis
2. Hierarchy: Manager → workers
3. Market: Task bidding system
```

CrewAI Implementation:

python

```
from crewai import Agent, Task, Crew

# Define agents
researcher = Agent(
    role="Research Specialist",
    goal="Find latest information",
    backstory="""Expert researcher with access to real-time data..."""
)

writer = Agent(
    role="Content Strategist",
    goal="Create compelling narratives",
    backstory="10+ years in digital marketing..."
)

# Define tasks
research_task = Task(
    description="Research AI agent frameworks 2024",
    agent=researcher
)

write_task = Task(
    description="Write executive summary",
    agent=writer,
    context=[research_task]
)

# Orchestrate
crew = Crew(agents=[researcher, writer], tasks=[research_task, write_task])
result = crew.kickoff()
```

AutoGen Multi-Agent Conversation:

text

GroupChat: Dynamic agent participation

Human Proxy: Real-time intervention

Section 6: Evaluation Frameworks and Production Deployment

Agent Evaluation Metrics:

text

1. Success Rate: Task completion

2. Efficiency: Steps to solution

3. Robustness: Failure recovery

4. Cost: Token budget

AgentBench Suite:

text

WebShop, WebArena, GAIA, SWE-Bench

Production Agent Architecture:

text

Load Balancer → Agent Router → Memory → Planner → Executor → Monitor

Safety and Guardrails:

text

1. Action approval workflows

2. Cost limits per step

3. Human-in-loop for high-stakes

4. Sandboxed tool execution

Section 7: Case Studies and Advanced Projects

Case Study: Devin (Cognition Labs):

text

Software engineering agent

End-to-end: Plan → Code → Test → Deploy

95th percentile SWE-bench

Capstone Project: Enterprise Automation Agent

text

1. Corporate tools integration (Jira, Slack, GitHub)

2. Multi-agent workflow

3. Production monitoring dashboard

4. Cost/performance optimization

Research Frontier: Agents in the Wild

text

1. Physical robots + agents

2. Economy simulations (Generative Agents)

3. Scientific discovery agents